代码原理文档

这个文档记录项目核心机制的实现原理，帮助理解代码为什么这样设计。

---

1. Slash 命令实现机制

1.1 整体架构

```
┌──────────────────────────────────────────────────────────────────┐
│                        AgentLoop.run()                           │
│  ┌──────────────────────────────────────────────────────────┐    │
│  │                 收到用户消息 (InboundMessage)                │    │
│  └──────────────────────────────────────────────────────────┘    │
│                            │                                      │
│                            ▼                                      │
│  ┌──────────────────────────────────────────────────────────┐    │
│  │                CommandRouter.is_priority(text)              │    │
│  │                判断是否为优先命令 (/stop, /restart, /status) │    │
│  └──────────────────────────────────────────────────────────┘    │
│                            │                                      │
│                            │                                      │
│              ┌─────────────┴─────────────┐                        │
│              │                           │                        │
│              ▼                           ▼                        │
│   ┌──────────────────┐     ┌──────────────────────┐              │
│   │  dispatch_priority │     │   普通消息处理流程     │              │
│   │  (/stop, /restart) │     │  (发送给 LLM 处理)      │              │
│   └──────────────────┘     └──────────────────────┘              │
│            │                                                      │
│            ▼                                                      │
│   ┌──────────────────┐                                            │
│   │ cmd_stop / cmd_restart │                                      │
│   │   具体命令处理函数      │                                      │
│   └──────────────────┘                                            │
└──────────────────────────────────────────────────────────────────┘
```

1.2 核心组件

命令路由器

位置: `nanobot/command/router.py`

```
class CommandRouter:
    def __init__(self) -> None:
        self._priority: dict[str, Handler] = {}      # 优先命令（立即执行，不等待锁）
        self._exact: dict[str, Handler] = {}         # 精确匹配
        self._prefix: list[tuple[str, Handler]] = [] # 前缀匹配
        self._interceptors: list[Handler] = []       # 拦截器（兜底）
```

三层匹配机制:

1. priority → 最高优先级，在获取锁之前执行（如 `/stop` 需要立即响应）
2. exact → 精确匹配
3. prefix → 前缀匹配（如 `/team xxx`）

4. interceptors → 拦截器兜底

命令上下文

```
@dataclass
class CommandContext:
    msg: InboundMessage      # 原始消息
    session: Session | None  # 当前会话
    key: str                 # 会话标识
    raw: str                 # 原始命令文本
    args: str = ""           # 命令参数
    loop: Any = None         # AgentLoop 引用
```

内置命令处理器

位置: `nanobot/command/builtin.py`

命令注册:

```
def register_builtin_commands(router: CommandRouter) -> None:
    router.priority("/stop", cmd_stop)      # 优先级
    router.priority("/restart", cmd_restart)
    router.priority("/status", cmd_status)
    router.exact("/new", cmd_new)           # 精确匹配
    router.exact("/help", cmd_help)
```

以 `/stop` 为例:

```
async def cmd_stop(ctx: CommandContext) -> OutboundMessage:
    """Cancel all active tasks and subagents for the session."""
    loop = ctx.loop
    msg = ctx.msg
    tasks = loop._active_tasks.pop(msg.session_key, [])  # 取出活跃任务
    cancelled = sum(1 for t in tasks if not t.done() and t.cancel())  # 取消
    # ... 等待任务完成 ...
    sub_cancelled = await loop.subagents.cancel_by_session(msg.session_key)  # 取消子代理
    total = cancelled + sub_cancelled
    content = f"Stopped {total} task(s)." if total else "No active task to stop."
    return OutboundMessage(channel=msg.channel, chat_id=msg.chat_id, content=content)
```

1.3 消息处理流程

命令注册

位置: `nanobot/agent/loop.py:128-129`

```
self.commands = CommandRouter()
register_builtin_commands(self.commands)  # 注册内置命令
```

命令分发

在 `AgentLoop.run()` 中（简化版）:

```
# 1. 检查是否为优先命令
if self.commands.is_priority(text):
    ctx = CommandContext(msg=msg, session=session, key=key, raw=text, loop=self)
    return await self.commands.dispatch_priority(ctx)

# 2. 普通命令分发（在锁内）
ctx = CommandContext(...)
result = await self.commands.dispatch(ctx)
if result:
```

```
    return result  # 命令已处理，直接返回

# 3. 不是命令，交给 LLM 处理
await self._process_with_llm(msg, session)
```

1.4 关键设计点

设计	原因
priority 级别	`/stop` 需要立即响应，不能等待锁
CommandContext dataclass	统一传递命令所需的所有上下文
返回 OutboundMessage	命令处理结果统一为出站消息格式
三层匹配	支持精确命令、前缀命令、拦截器等多种场景

1.5 文件关系

```
nanobot/
├── command/
│   ├── router.py      # CommandRouter 类（路由逻辑）
│   ├── builtin.py     # 内置命令实现（cmd_stop 等）
│   └── __init__.py    # 导出
└── agent/
    └── loop.py        # 注册和使用命令路由
```

1.6 下一步建议

1. 阅读 `nanobot/command/router.py` 完整理解路由逻辑
2. 阅读 `nanobot/agent/loop.py` 查看命令分发调用点
3. 查看 `nanobot/bus/events.py` 理解 `InboundMessage` / `OutboundMessage` 结构

---

2. CLI 命令实现机制

2.1 整体架构

```
┌──────────────────────────────────────────────────────────────┐
│                      pyproject.toml                          │
│  [project.scripts]                                           │
│  nanobot = "nanobot.cli.commands:app"  ← 入口点              │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│                   nanobot/cli/commands.py                    │
│                                                              │
│  app = typer.Typer(name="nanobot", ...)                      │
│                                                              │
│  @app.command()                                              │
│  def agent(...):    # → nanobot agent                        │
│                                                              │
│  @app.command()                                              │
│  def gateway(...):  # → nanobot gateway                      │
│                                                              │
```

```
│  @app.command()                                                  │
│  def status(...):   # → nanobot status                           │
│                                                                  │
│  @app.command()                                                  │
│  def onboard(...):  # → nanobot onboard                         │
└──────────────────────────────────────────────────────────────────┘
```

2.2 核心组件

入口定义

位置: `pyproject.toml`

```
[project.scripts]
nanobot = "nanobot.cli.commands:app"
```

解释：告诉 pip，当用户输入 `nanobot` 命令时，调用 `nanobot.cli.commands` 模块里的 `app` 对象。

Typer 应用

位置: `nanobot/cli/commands.py`

```
import typer

app = typer.Typer(
    name="nanobot",
    context_settings={"help_option_names": ["-h", "--help"]},
    help="🐈 nanobot - Personal AI Assistant",
    no_args_is_help=True,
)
```

命令定义

```
@app.command()
def agent(
    message: str = typer.Option(None, "--message", "-m", help="Message to send"),
    session_id: str = typer.Option("cli:direct", "--session", "-s"),
    workspace: str | None = typer.Option(None, "--workspace", "-w"),
):
    """Start CLI chat session."""
    # ...

@app.command()
def gateway(
    port: int | None = typer.Option(None, "--port", "-p"),
    workspace: str | None = typer.Option(None, "--workspace", "-w"),
):
    """Start the gateway server."""
    # ...

@app.command()
def status():
    """Show nanobot status."""
    # ...

@app.command()
def onboard(
    workspace: str | None = typer.Option(None, "--workspace", "-w"),
):
    """Initialize workspace and config."""
    # ...
```

2.3 执行效果

```
nanobot agent          # 启动 CLI 对话
nanobot gateway        # 启动网关
nanobot status         # 显示状态
nanobot onboard        # 初始化工作区
nanobot --help         # 显示所有命令
nanobot agent --help   # 显示 agent 命令帮助
```

2.4 typer 工作原理

```
@app.command()
def hello(name: str):
    print(f"Hello {name}!")
```

等价于:

```
def hello(name: str):
    print(f"Hello {name}!")

app.command()(hello)
```

typer 自动做了什么:

1. 解析函数签名 (`name: str` 参数)
2. 自动生成 `--name` 选项
3. 自动生成帮助信息
4. 解析命令行参数并调用函数

2.5 关键设计点

设计	原因
`typer.Typer()`	创建 CLI 应用容器，统一管理命令
`@app.command()`	把函数变成子命令，函数名即命令名
`typer.Option()`	定义命令行选项（可选参数）
`typer.Argument()`	定义命令行参数（必填）
`[project.scripts]`	让 pip 安装后生成可执行文件

2.6 文件关系

```
nanobot/
├── cli/
│   ├── commands.py    # CLI 命令定义
│   ├── stream.py      # 输出流渲染
│   ├── onboard.py     # 工作区初始化
│   └── models.py      # 模型列表
└── pyproject.toml        # 定义入口点
```

2.7 下一步建议

1. 读 `nanobot/cli/commands.py` 看完整的命令实现
2. 读 `pyproject.toml` 的 `[project.scripts]` 配置

3. 自己写一个简单命令练习:

```
@app.command()
def hello(name: str = typer.Argument("World")):
    print(f"Hello {name}!")
```

---

3. 核心循环实现机制

3.1 整体架构

nanobot 核心循环采用**两层循环**架构：外层循环负责消息接收和分发，内层循环负责 LLM 调用 + 工具执行的迭代。

```
┌──────────────────────────────────────────────────────────┐
│  外层循环 (AgentLoop.run)          loop.py:276            │
│  while self._running:                                     │
│    msg = bus.consume_inbound()       ← 阻塞等待消息        │
│    asyncio.create_task(dispatch(msg))  ← 异步分发          │
└──────────────────────────────────────────────────────────┘
                            |
                            |
                            v
┌──────────────────────────────────────────────────────────┐
│  内层循环 (AgentRunner.run)         runner.py:58           │
│  for iteration in range(max_iterations):   ← 默认 40 次   │
│    response = LLM(messages)                               │
│    if response.has_tool_calls:                            │
│      执行工具 → 追加结果 → continue                         │
│    else:                                                  │
│      break (最终回答)                                       │
└──────────────────────────────────────────────────────────┘
```

3.2 关键文件

文件	职责
`nanobot/agent/loop.py`	外层循环：消息消费、会话管理、分发
`nanobot/agent/runner.py`	内层循环：LLM 调用 + 工具调用迭代
`nanobot/agent/hook.py`	Hook 接口，注入到 runner 中
`nanobot/bus/queue.py`	MessageBus（asyncio 队列）
`nanobot/agent/tools/registry.py`	工具注册和执行分发

3.3 外层循环 — `loop.py:276`

```
async def run(self):
    self._running = True
    while self._running:
        msg = await asyncio.wait_for(
            self.bus.consume_inbound(), timeout=1.0)  # 最多等 1 秒
        task = asyncio.create_task(self._dispatch(msg))  # 非阻塞分发
```

- **无限循环**，靠 `_running` 标志控制退出

- 从 `MessageBus` 阻塞消费消息（1 秒超时检查退出标志）
- 每条消息创建独立的 **asyncio Task** 处理，保持响应性
- 同一会话内用 `asyncio.Lock` 保证消息串行处理（`loop.py:310`）

3.4 内层循环 — `runner.py:58`

```
for iteration in range(spec.max_iterations):   # 默认 40 次
    response = await self.provider.chat_with_retry(...)  # 调 LLM

    if response.has_tool_calls:
        # 1) 追加 assistant 消息（含 tool_calls）
        # 2) 执行工具 (_execute_tools)
        # 3) 追加 tool 结果消息
        continue                    # → 下一轮迭代

    # 无 tool_calls = 最终回答
    final_content = clean
    break                           # → 退出
else:
    stop_reason = "max_iterations"  # 40 次用完
```

这是经典的 **ReAct 模式**：每次 LLM 返回后判断是否需要调工具，需要就执行后继续，不需要就结束。

3.5 工具调用流程

```
LLM 返回 tool_calls
    |
    v
runner.py:107  追加 assistant 消息到历史
    |
    v
runner.py:117  _execute_tools()  ──→  _run_tool()  ──→  ToolRegistry.execute()
    |                                                   查找工具、校验参数、调用
    v
runner.py:128  追加 tool 结果消息到历史
    |
    v
continue → 下一轮 LLM 调用（带着工具结果）
```

工具执行支持两种模式：

- **顺序执行**（默认）：工具一个一个执行
- **并发执行**（`spec.concurrent_tools=True`）：通过 `asyncio.gather` 并行执行

单个工具的执行路径（`runner.py:203` → `tools/registry.py:38`）：

```
# registry.py:38-59
def execute(self, name: str, arguments: dict):
    tool = self._tools[name]          # 按名查找工具
    params = tool.validate(arguments)  # 按 JSON Schema 校验并转换参数
    return tool.execute(**params)      # 调用工具
```

3.6 终止条件

内层循环有四种终止方式：

条件	位置	stop_reason
LLM 返回纯文本（无工具调用）	`runner.py:138`	`completed`
LLM 返回错误	`runner.py:142`	`error`
工具执行致命错误（`fail_on_tool_error=True`）	`runner.py:121`	`tool_error`
达到最大迭代次数（40）	`runner.py:162`	`max_iterations`

外层循环通过 `AgentLoop.stop()`（`loop.py:384`）设置 `_running = False` 退出，1 秒超时保证至少每秒检查一次退出标志。

3.7 完整数据流

```
Channel (Telegram/Discord/CLI)
  |
  v
bus.publish_inbound(InboundMessage)
  |
  v
AgentLoop.run()  →  consume_inbound()
  |
  v
_dispatch()  →  _process_message()  （会话/上下文构建）
  |
  v
AgentRunner.run()  →  LLM 调用 ↔ 工具执行 循环
  |
  v
bus.publish_outbound(OutboundMessage)
  |
  v
Channel 投递给用户
```

3.8 Hook 机制

`AgentHook`（`hook.py:27`）在 runner 循环的关键节点提供回调：

- `before_execute_tools(context)` — 工具执行前，用于发送进度提示、日志记录
- `_LoopHook`（`loop.py:244-255`）是 `AgentHook` 的具体实现，将工具调用信息反馈给用户

3.9 关键设计点

设计	原因
两层循环分离	外层管消息调度，内层管 LLM 迭代，职责清晰
asyncio Task 分发	消息处理不阻塞外层循环，支持多会话并发
会话锁（asyncio.Lock）	同一会话的消息串行处理，避免历史混乱
for-else 检测超限	防止无限循环，40 次上限兜底
工具结果不抛异常	错误作为字符串返回给 LLM，让 LLM 自行决定下一步

3.10 下一步建议

1. 阅读 `nanobot/agent/runner.py` 完整理解 LLM 调用和工具执行逻辑
2. 阅读 `nanobot/agent/loop.py` 理解 `_process_message` 如何构建上下文
3. 阅读 `nanobot/agent/tools/registry.py` 理解工具注册和参数校验

---

4. ExecTool：Shell 命令执行机制

4.1 整体架构

```
┌──────────────────────────────────────────────────────────┐
│  AgentLoop / SubAgent                                    │
│    注册 ExecTool (loop.py:246 / subagent.py:119)         │
└──────────────────────────────────────────────────────────┘
                            │
                            │
                            v
┌──────────────────────────────────────────────────────────┐
│  ToolRegistry.execute(name="exec", params={...})         │
│    registry.py:38                                        │
│    1. tool.cast_params()      ← 参数类型转换             │
│    2. tool.validate_params()  ← 参数校验                 │
│    3. tool.execute(**params)  ← 调到 ExecTool.execute()  │
└──────────────────────────────────────────────────────────┘
                            │
                            │
                            v
┌──────────────────────────────────────────────────────────┐
│  ExecTool.execute()              shell.py:81             │
│                                                          │
│  1. _guard_command()             ← 安全检查              │
│     ├─ deny_patterns            黑名单                   │
│     ├─ allow_patterns           白名单                   │
│     ├─ contains_internal_url    内网 URL 检测            │
│     └─ restrict_to_workspace    路径限制                 │
│                                                          │
│  2. asyncio.create_subprocess_shell()  ← 真正执行命令    │
│                                                          │
│  3. asyncio.wait_for(communicate())   ← 等待结果+超时    │
│                                                          │
│  4. 拼装 stdout/stderr/exit code     ← 输出处理          │
│     └─ 超长时头尾截断 (10000 字符)                       │
└──────────────────────────────────────────────────────────┘
```

4.2 关键文件

文件	职责
`nanobot/agent/tools/shell.py`	ExecTool，执行 shell 命令
`nanobot/agent/tools/base.py`	Tool 抽象基类
`nanobot/agent/tools/registry.py`	工具注册与调度
`nanobot/config/schema.py`	ExecToolConfig 配置
`nanobot/security/network.py`	contains_internal_url 内网检测

4.3 入口：`execute()` 方法 — `shell.py:81`

```
async def execute(
    self, command: str, working_dir: str | None = None,
    timeout: int | None = None, **kwargs: Any,
) -> str:
```

执行流程分四步：

第一步 — 确定工作目录 (shell.py:85)

```
cwd = working_dir or self.working_dir or os.getcwd()
```

优先级：调用时传入 > 构造时传入 > 当前目录。

第二步 — 安全检查 (shell.py:86-88)

```
guard_error = self._guard_command(command, cwd)
if guard_error:
    return guard_error
```

不通过就直接返回错误字符串，**不执行命令**。

第三步 — 执行命令 (shell.py:97-103)

```
process = await asyncio.create_subprocess_shell(
    command,
    stdout=asyncio.subprocess.PIPE,
    stderr=asyncio.subprocess.PIPE,
    cwd=cwd,
    env=env,
)
```

这是**真正的执行点**。用 `asyncio.create_subprocess_shell` 创建子进程，stdout/stderr 都通过管道捕获。

第四步 — 等待结果，处理超时 (shell.py:106-122)

```
stdout, stderr = await asyncio.wait_for(
    process.communicate(),
    timeout=effective_timeout,
)
```

`asyncio.wait_for` 包裹 `communicate()`，超时后 `process.kill()` 杀进程。

4.4 安全守卫：`_guard_command()` — `shell.py:153`

四层安全检查：

层	检查内容	代码位置
1	**deny_patterns** — 黑名单匹配 `rm -rf`、`format`、`shutdown` 等危险命令	:158-160
2	**allow_patterns** — 白名单模式，如果设置了则只允许匹配的命令	:162-164
3	**contains_internal_url** — 检测命令中是否包含内网 URL（如 `localhost`、`192.168.*`）	:166-168
4	**restrict_to_workspace** — 限制路径不能超出工作目录（防止路径穿越 `../`）	:170-183

返回 `None` 表示检查通过，返回字符串表示错误信息。

4.5 输出处理 — `shell.py:124`

- stdout 和 stderr 分别解码拼接
- 始终追加 `Exit code: N`
- 输出超过 `_MAX_OUTPUT` (10000 字符) 时做**头尾截断**：保留前半 + 后半，中间标注省略了多少字符

4.6 注册与调用关系

`ExecTool` 在 `AgentLoop`（loop.py:246）和 `SubAgent`（subagent.py:119）中按需注册：

```
# loop.py:246
self.tools.register(ExecTool(
    timeout=self.exec_config.timeout,
    working_dir=working_dir,
    deny_patterns=self.exec_config.deny_patterns,
    ...
))
```

注册后，当 LLM 返回 `function_call` 指定 `name="exec"` 时，`ToolRegistry.execute()` 找到这个工具，先做参数转换和校验，再调 `ExecTool.execute()`。

4.7 完整调用链

```
LLM 返回 function_call (name="exec")
  |
  v
runner.py:117  _execute_tools()  ──→  ToolRegistry.execute()
  |                                     ├─ cast_params()
  |                                     ├─ validate_params()
  |                                     └─ ExecTool.execute()
  |                                          ├─ _guard_command()  安全检查
  |                                          ├─ asyncio.create_subprocess_shell()  执行
  |                                          └─ asyncio.wait_for(communicate())  等待
  |
  v
runner.py:128  追加 tool 结果消息到历史 → continue → 下一轮 LLM 调用
```

4.8 关键设计点

设计	原因
四层安全检查	防止模型执行危险命令、访问内网、越权操作文件
asyncio.create_subprocess_shell	异步子进程，不阻塞事件循环
asyncio.wait_for + process.kill	超时自动终止，防止命令卡死
头尾截断输出	保留首尾关键信息，避免撑爆 LLM 上下文窗口
ToolRegistry 统一调度	所有工具共享参数转换、校验、错误提示流程

4.9 下一步建议

1. 读 `nanobot/security/network.py`，了解 `contains_internal_url` 如何检测内网地址
2. 读 `nanobot/config/schema.py:133` 的 `ExecToolConfig`，了解配置项如何从 YAML 传入 ExecTool
3. 读 `nanobot/agent/loop.py:240-250`，了解 ExecTool 注册时的条件判断和参数传递

---

5. Heartbeat 心跳巡检机制

5.1 整体架构

Heartbeat 是 nanobot 的定时自唤醒机制——让 agent 在没有用户消息时，也能周期性地主动检查是否有待办任务并执行。

```
用户在 HEARTBEAT.md 中写入周期性任务
        │
        ▼
┌──────────────────────────────────┐
│  _run_loop (每 30min 触发一次)    │
│           │                      │
│           ▼                      │
│  Phase 1: _decide (决策)         │
│  读取 HEARTBEAT.md，送给 LLM     │
│  LLM 通过 tool call 返回:        │
│    skip → 无任务，跳过            │
│    run  → 有任务，进入 Phase 2    │
│           │                      │
│           ▼ (run)                │
│  Phase 2: on_execute (执行)      │
│  走完整 agent loop 执行任务       │
│           │                      │
│           ▼                      │
│  evaluate_response (评估)        │
│  再问 LLM: 结果值得通知用户吗?    │
│    yes → on_notify 推送到频道     │
│    no  → 静默，不打扰用户         │
└──────────────────────────────────┘
```

5.2 关键文件

文件	职责
`nanobot/heartbeat/service.py`	核心服务，管理定时循环、决策、执行
`nanobot/config/schema.py:91`	`HeartbeatConfig`：是否启用、间隔秒数（默认 30min）、保留消息数
`nanobot/templates/HEARTBEAT.md`	任务清单文件，用户在这里写周期性任务
`nanobot/utils/evaluator.py`	执行后评估，决定是否通知用户
`nanobot/cli/commands.py:697-744`	heartbeat 在 gateway 启动时被组装并接入 agent 和 channel

5.3 两阶段决策流程

Phase 1: LLM 决策 — `service.py:87`

```
async def _decide(self, content: str) -> tuple[str, str]:
    response = await self.provider.chat_with_retry(
        messages=[
            {"role": "system", "content": "You are a heartbeat agent. Call the heartbeat tool to
report your decision."},
            {"role": "user", "content": (
```

```
                f"Current Time: {current_time_str(self.timezone)}\n\n"
                "Review the following HEARTBEAT.md and decide whether there are active tasks.\n\n"
                f"{content}"
            )},
        ],
        tools=_HEARTBEAT_TOOL,
        model=self.model,
    )
    args = response.tool_calls[0].arguments
    return args.get("action", "skip"), args.get("tasks", "")
```

不是简单的正则匹配，而是把 `HEARTBEAT.md` 内容连同当前时间一起发给 LLM，让 LLM 通过一个虚拟 tool call（`action: skip/run`）判断是否有活跃任务。这避免了自由文本解析的不可靠性。

Phase 2: 执行 + 评估 — `service.py:145`

```
async def _tick(self) -> None:
    content = self._read_heartbeat_file()   # 每次重新读取 HEARTBEAT.md
    action, tasks = await self._decide(content)
    if action != "run":
        return
    response = await self.on_execute(tasks)  # 走完整 agent loop
    should_notify = await evaluate_response(response, tasks, ...)  # 评估是否通知
    if should_notify:
        await self.on_notify(response)  # 推送到用户频道
```

5.4 Heartbeat vs Cron 对比

```
┌──────────────────────────────────────────────────────┐
│                                                      │
│   Cron（定时任务）                Heartbeat（心跳巡检）    │
│   ──────────────                  ────────────────────   │
│   用户说"每2分钟提醒我喝水"     agent 每30min自己醒来    │
│         │                              │                 │
│         ▼                              ▼                 │
│   精确的时间调度                  固定间隔轮询             │
│   (cron表达式/ every N分钟)       (默认30分钟)            │
│         │                              │                 │
│         ▼                              ▼                 │
│   到点就执行                      先读HEARTBEAT.md        │
│   不需要判断"要不要做"            再让LLM判断有无任务      │
│         │                              │                 │
│         ▼                              ▼                 │
│   直接执行+通知用户                有任务才执行           │
│                                                          │
└──────────────────────────────────────────────────────┘
```

	Cron	Heartbeat
触发方式	精确时间调度（类似 crontab）	固定间隔轮询（默认 30min）
任务来源	用户在对话中创建（如"每2分钟提醒我喝水"）	用户在 `HEARTBEAT.md` 文件中手写
是否需要判断	不需要，到点就执行	需要 LLM 判断 HEARTBEAT.md 里有没有活跃任务
适用场景	明确的定时提醒/定时任务	周期性巡检、状态检查等模糊场景
持久化	`cron/jobs.json`	`HEARTBEAT.md` 文件

5.5 重要特性：任务不会自动标记完成

每次 tick 都会重新读取 `HEARTBEAT.md`，不会标记"已执行"。

```
Cron Job:     到点执行 → 完成就结束 → 等下一次调度
Heartbeat:    每次醒来都读同一份列表 → 全部重新检查 → 循环往复
```

用户如果想停掉某个任务，需要手动把内容从 Active Tasks 移到 Completed 区域，或者直接删除。Heartbeat 本身不会自动帮你划掉。

这也呼应了它的定位——适合**持续性的巡检任务**（"定期看服务有没有挂"），不适合**一次性任务**（"提醒我下午3点开会"）。

5.6 典型使用场景

状态监控类

```
## Active Tasks
- [ ] 检查线上服务是否正常
- [ ] 查看今天的 CI 构建有没有失败
- [ ] 监控磁盘空间是否快满了
```

信息聚合类

```
## Active Tasks
- [ ] 汇总今天 GitHub 上收到的新 issue
- [ ] 检查有没有未回复的重要邮件
- [ ] 看看今天日历上有没有即将到来的会议
```

习惯跟踪类

```
## Active Tasks
- [ ] 提醒我今天还没写日报
- [ ] 检查本周代码 review 是否还有积压
```

Heartbeat 的优势在于：

- **低门槛**：用户只需编辑一个 Markdown 文件，不用学 cron 表达式
- **弹性判断**：LLM 决定"这次要不要执行"，不是机械地到点就跑
- **批量处理**：一个文件里写多个任务，一次心跳全部检查
- **自然语言任务**：不用严格格式，LLM 能理解"检查服务状态"这种描述
- **防打扰**：`evaluate_response` 环节会再问 LLM 判断结果是否值得通知用户

5.7 防打扰机制 — `evaluator.py`

Phase 2 执行完之后，`evaluate_response()` 会再次调 LLM 判断结果是否值得通知用户：

```
# evaluator.py
_SYSTEM_PROMPT = (
    "You are a notification gate for a background agent. "
    "Notify when the response contains actionable information, errors, "
    "completed deliverables, or anything the user explicitly asked to "
    "be reminded about.\n\n"
    "Suppress when the response is a routine status check with nothing "
    "new, a confirmation that everything is normal, or essentially empty."
)
```

如果只是"一切正常"的例行检查，会静默处理，不打扰用户。失败时默认通知（`return True`），避免重要消息被静默丢弃。

5.8 关键设计点

设计	原因
LLM 驱动决策（虚拟 tool call）	避免自由文本解析不可靠，结构化输出 `skip/run`
两阶段分离	先轻量判断有无任务，再重量级执行，省 token
evaluate_response 评估通知	防止例行检查结果频繁打扰用户
keep_recent_messages=8	控制 heartbeat session 历史长度，防止无限增长
on_execute / on_notify 回调	解耦 heartbeat 服务与 agent loop、channel 的依赖
失败时默认通知	`evaluate_response` 出错时 `return True`，不丢重要消息

5.9 下一步建议

1. 读 `tests/agent/test_heartbeat_service.py`，看各种场景的测试用例（skip / run / 执行失败 / 静默通知等）
2. 读 `nanobot/cli/commands.py:690-744`，看 heartbeat 如何在 gateway 启动时被组装
3. 读 `nanobot/utils/evaluator.py`，理解通知评估的完整逻辑
4. 对比读 `nanobot/cron/service.py`，理解 Cron 与 Heartbeat 的实现差异